

Project: Vehicle Insurance Policy and Claims Management System

1. Overview

The **Vehicle Insurance Policy and Claims Management System** is a web-based application designed to help customers manage vehicle insurance policies and file claims. It includes functionalities for policy creation, customer registration, vehicle details management, claim filing, and premium calculation.

This application uses the **MVC architecture** and can be implemented with **Java (Spring MVC)** or **.NET (ASP.NET Core MVC)** frameworks, ensuring cross-platform compatibility.

2. Assumptions

- Customers can manage multiple vehicles under a single account.
- The system will calculate premiums based on vehicle type, age, and coverage options.
- Insurance administrators can process and approve/reject claims.
- The database is designed for scalability with relationships between customers, vehicles, policies, and claims.

3. Module-Level Design

3.1 Customer Management Module

Purpose: Manages customer registration and profile updates.

- **Controller:**
 - CustomerController
 - registerCustomer()
 - updateCustomerProfile()
 - getCustomerDetails()
- **Service:**
 - CustomerService
 - Handles business logic for profile creation and updates.
- **Model:**
 - **Customer Entity**
 - Attributes:
 - customerId (PK)
 - name
 - email
 - phone
 - address

3.2 Vehicle Management Module

Purpose: Allows customers to add, view, and update vehicle details.

- **Controller:**

- VehicleController
 - addVehicle()
 - getVehicleDetails()
 - updateVehicleDetails()
- **Service:**
 - VehicleService
 - Validates and processes vehicle data.
- **Model:**
 - **Vehicle Entity**
 - Attributes:
 - vehicleId (PK)
 - customerId (FK)
 - registrationNumber
 - make
 - model
 - yearOfManufacture
 - vehicleType (CAR, BIKE, TRUCK)

3.3 Policy Management Module

Purpose: Handles policy creation, renewal, and premium calculations.

- **Controller:**
 - PolicyController
 - createPolicy()
 - getPolicyDetails()
 - renewPolicy()
- **Service:**
 - PolicyService
 - Implements business rules for policy creation and renewal.
- **Model:**
 - **Policy Entity**
 - Attributes:
 - policyId (PK)
 - vehicleId (FK)
 - policyNumber
 - coverageAmount
 - premiumAmount
 - startDate
 - endDate
 - policyStatus (ACTIVE, EXPIRED)

3.4 Claims Management Module

Purpose: Manages claim submission and processing.

- **Controller:**

- ClaimController
 - fileClaim()
 - getClaimStatus()
 - processClaim()
- **Service:**
 - ClaimService
 - Validates claims and processes approvals or rejections.
- **Model:**
 - **Claim Entity**
 - Attributes:
 - claimId (PK)
 - policyId (FK)
 - claimAmount
 - claimReason
 - claimDate
 - claimStatus (SUBMITTED, APPROVED, REJECTED)

3.5 Reporting Module

Purpose: Generates reports for customers and administrators.

- **Controller:**
 - ReportController
 - generateClaimReport()
 - generatePolicyReport()
- **Service:**
 - ReportService
 - Prepares data for reports in various formats (PDF, Excel).
- **Model:**
 - Aggregates data from other modules.

4. Database Schema

4.1 Table Definitions

1. Customer Table

```
CREATE TABLE Customer (
  customerId INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100),
  phone VARCHAR(15),
  address TEXT
);
```

2. Vehicle Table

```
CREATE TABLE Vehicle (
  vehicleId INT AUTO_INCREMENT PRIMARY KEY,
```

```
customerId INT,  
registrationNumber VARCHAR(20),  
make VARCHAR(50),  
model VARCHAR(50),  
yearOfManufacture INT,  
vehicleType ENUM('CAR', 'BIKE', 'TRUCK'),  
FOREIGN KEY (customerId) REFERENCES Customer(customerId)  
);
```

3. Policy Table

```
CREATE TABLE Policy (  
  policyId INT AUTO_INCREMENT PRIMARY KEY,  
  vehicleId INT,  
  policyNumber VARCHAR(20),  
  coverageAmount DECIMAL(10, 2),  
  premiumAmount DECIMAL(10, 2),  
  startDate DATE,  
  endDate DATE,  
  policyStatus ENUM('ACTIVE', 'EXPIRED'),  
  FOREIGN KEY (vehicleId) REFERENCES Vehicle(vehicleId)  
);
```

4. Claim Table

```
CREATE TABLE Claim (  
  claimId INT AUTO_INCREMENT PRIMARY KEY,  
  policyId INT,  
  claimAmount DECIMAL(10, 2),  
  claimReason TEXT,  
  claimDate DATE,  
  claimStatus ENUM('SUBMITTED', 'APPROVED', 'REJECTED'),  
  FOREIGN KEY (policyId) REFERENCES Policy(policyId)  
);
```

5. Local Deployment Details

1. Environment Setup:

- Install MySQL or SQL Server and create the database schema using the provided SQL scripts.
- Set up the development environment with either JDK (Java) or .NET SDK (ASP.NET Core).
- Configure application settings for database connections.

2. Steps to Deploy Locally:

- Clone the project repository.
- Build the project using Maven (Java) or Visual Studio (ASP.NET Core).
- Start the application server (Tomcat for Java or IIS/Kestrel for .NET).
- Access the application at <http://localhost:8080> (Java) or <http://localhost:5000> (.NET).

6. Conclusion

The **Vehicle Insurance Policy and Claims Management System** offers a user-friendly interface for managing vehicle policies and claims. Its modular design ensures scalability, maintainability, and compatibility with enterprise-grade platforms. The system effectively addresses the key challenges of the vehicle insurance domain.